OVERVIEW

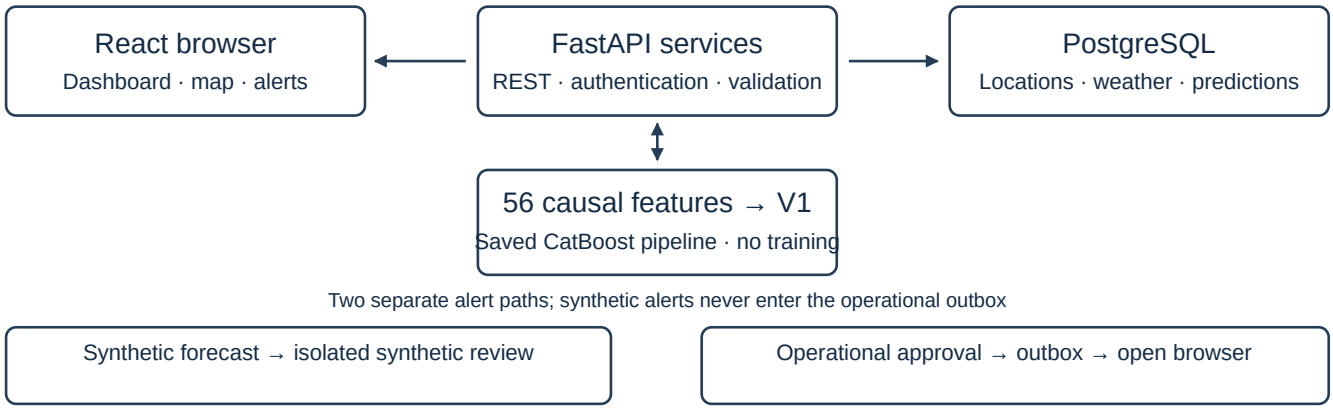
Five Functionalities Demonstration Guide

Fiji Flood Risk Prediction and Community Alert System · Project evaluation

A university proof of concept for displaying flood-risk information, managing weather records and reviewing alerts. The application currently serves a **synthetic Fiji CatBoost V1** model. It is not a validated real-world Fiji forecasting service.

Approved functionality	Verified core / remaining scope
1 · Flood Risk Dashboard	API-backed views and analytics; personalization is partial.
2 · Weather Data Management	Validated storage and retrieval; real-provider acquisition is unconfigured.
3 · AI Flood Risk Prediction	Working synthetic inference; genuine Fiji forecasting remains unavailable.
4 · Flood Mapping	12-location point map and admin APIs; no PostGIS flood-area layer.
5 · Alerts & System Testing	Lifecycle and notification pipeline tested; desktop delivery not observed.

Architecture and data flow



What this guide verifies

Source and artifacts were inspected alongside the supplied **SRS_Group13.pdf** and **SAD_Group13-1.pdf**. Fresh automated tests, read-only PostgreSQL queries, live GET requests and public-page browser checks were used. No application changes, migrations, retraining, new alerts or Git actions were performed.

Evidence labels: **Verified** inspected and exercised; **Tested** automated coverage; **Conditional** needs a permitted account, setting or external service. Some older README sections describe an earlier prototype; source and current evidence take precedence.

01 · FRONTEND

Flood Risk Dashboard

Status: core views implemented; full SRS personalization is partial

Purpose and where to find it

The dashboard brings together stored flood-risk information and review work. It helps an administrator decide which locations and alerts need attention. Sign in at `/login`; MET administrators use `/admin/dashboard`, community users use `/dashboard`.

How it works

1. Login returns a JWT; `GET /api/auth/me` verifies the account and role.
2. React requests locations and alerts. Admin analytics request stored summaries and saved evaluation artifacts.
3. The browser renders counts, recent predictions and charts; viewing these pages does not train or generate a model.

Live demonstration

1. Use your own approved MET-admin account. Open **Dashboard**; show monitored locations, active/pending alerts and high-risk areas.
2. Open **Prediction analytics** (`/admin/analytics`). Compare “Genuine and sample” with “Synthetic research”; explain empty or one-point histories.
3. Open **Model performance** (`/admin/model-performance`). Select the V1 test partition and show its confusion matrix.
4. Open **Weather map** for actual stored weather readings. In **My overview**, show saved-location controls and notification history.

Success check and current limits

Verified: the registry has 12 locations in four divisions; the analytics service reads the V1 artifacts. Role protection passed tests; an unauthenticated browser was redirected to login. The current map prediction records are **samples**, not new V1 forecasts. Weather is shown in map detail/replay, not a complete weather dashboard. Saved-location management exists, but a fully personalized saved-location risk summary is incomplete. A signed-in admin browser walkthrough was not performed during this audit.

Technologies / APIs: React, TypeScript, Recharts, FastAPI, SQLAlchemy, PostgreSQL; `GET /api/locations`, `/api/alerts/active`, `/api/admin/analytics/predictions/overview`, `/predictions/trend` and `/models/performance` under that analytics prefix.

One-minute lecturer script

“This dashboard uses backend records rather than a fabricated local dataset. An approved administrator sees monitored locations, alert counts and recent predictions. The analytics pages separate synthetic research from genuine and sample records. The model-performance page shows saved evaluation results and does not retrain anything. Notice the source and age labels: a sample prediction is not a live model forecast. Community accounts also have saved locations and notification history. For weather measurements, I open the location card on the map. This demonstrates the integrated interface while being clear about the remaining personalization and real-data limitations.”

Source trail: `frontend/src/pages/{AdminDashboardPage,DashboardPage,AdminAnalyticsPage,AdminModelPerformancePage}.tsx`; `backend/app/services/analytics_service.py`. Tables: `locations`, `predictions`, `synthetic_predictions`, `alerts`.

02 · BACKEND

Weather Data Management

Status: storage/API workflow implemented; external acquisition is unconfigured

Purpose and how it works

Weather management keeps measurements traceable, consistent and available to other modules. It prevents missing or synthetic measurements from being mistaken for fresh genuine readings.

1. An admin ingestion API, simulator import or configurable polling worker supplies observations.
2. Pydantic checks location, timezone, finite ranges and rainfall accumulation periods. Source/location/timestamp identities prevent duplicates.
3. SQLAlchemy saves genuine and synthetic observations separately in PostgreSQL. Read APIs return units, source, missing fields and freshness.
4. The map detail card retrieves the selected location's latest synthetic observation.

Where and what to demonstrate

1. Open **Weather map → Nausori → Current weather**. Show the synthetic/stale badges and observation time.
2. In `http://127.0.0.1:8000/docs`, execute `GET /api/weather/latest` with `location_id=2`, `dataset=synthetic`.
3. Repeat with `dataset=genuine`: it returns `no_data`. Show history through `GET /api/weather/observations`.
4. Expand `POST /api/admin/weather/observations`: explain authorization and validation tests. No weather-entry form exists; do not submit invented values.

Current PostgreSQL storage	Verified rows
`weather_observations` · genuine	0
`demo_weather_observations` · canonical synthetic	2,100
`synthetic_observations` · full replay archive	525,888

Verification and limits

The canonical synthetic source is `synthetic-fiji-v1-weather`; its snapshot ends at **2024-07-10 00:00 UTC**. Tests cover invalid ranges, duplicates, source separation and feed retries. The worker is **disabled** and no provider URL is configured. Automatic Fiji MET acquisition and a manual fetch UI are unavailable. Real inputs need agreed access, mappings, units and gauge datums.

One-minute lecturer script

"Weather management begins with the source of a measurement. Each observation has a location, time, source and units. The backend validates it before storage and handles duplicate identities safely. Here the map reads an existing synthetic observation, and the stale label tells us it is historical. The genuine-weather request returns no data because a genuine feed is not configured. That is the correct result. The polling worker and authorized ingestion API are implemented, but we should not describe them as a connected Fiji MET service. Our next step would be an approved provider adapter and verified measurement conventions."

Source trail: FastAPI/Pydantic, SQLAlchemy/psycopg and PostgreSQL. `backend/app/api/routes/weather.py`; `schemas/weather.py`; `services/weather_service.py`, `weather_ingestion.py`; `frontend/src/locations/useLocationWeather.ts`; `backend/tests/test_weather*.py`.

03 · AI ENGINEERING

AI Flood Risk Prediction

Status: verified synthetic proof of concept; real-world prediction remains unvalidated

Purpose, dataset and model

The model estimates whether a **new simulated flood starts in the next 24 hours**. V1 uses 525,888 hourly observations for 12 existing Fiji locations, 2021–2025. Eligible labels contain 6,672 positives and 505,024 negatives (1.30% positive). Hidden states determine simulator events and are excluded from inputs.

How it works

1. Training builds 56 causal weather/history/geography features: 3/6/24/72/168-hour rainfall totals, upstream history, river/soil trends and missingness flags.
2. Models fit training only. Calibration, model choice and threshold choice use separate chronological validation blocks, with embargoes and location/storm holdouts.
3. Stored-weather inference reads 175 hourly records → saved imputer/scaler → unchanged CatBoost → probability and provisional risk tier. A separate API can persist provenance and queue isolated review.

V1 test · raw models	Precision	Recall	F1	PR-AUC
Logistic Regression	0.3735	0.1582	0.2222	0.2453
Random Forest	0.2738	0.1224	0.1692	0.1791
LightGBM	0.3137	0.2713	0.2909	0.2501
CatBoost · selected	**0.3442**	**0.2959**	**0.3182**	**0.2839**

Selected V1: threshold 0.44469; test Brier 0.0119; accuracy 98.42%; TP 348, FP 663, FN 828, TN 92,565. Event recall 87.76% across 49 events, median detected-event lead 9 h. High accuracy alone is misleading: the majority baseline has 98.75% accuracy and zero recall.

Demonstrate and verify

1. Open **Model performance** for comparison and metrics. V2/V3 and foundation-model studies are research-only; V1 remains deployed.
2. Open **Synthetic replay** (`/simulation`), enter **2024-07-30T01:00** UTC and click **Load synthetic replay**. Nausori shows 72.8%, explicitly synthetic.
3. Explain `POST /api/predictions/generate` in Swagger: it requires an approved MET admin and explicit research/replay flags. The current canonical snapshot supports July 10, not July 30.

One-minute lecturer script

“Our deployed model is CatBoost V1, selected using validation data. It learns from simulated weather and flood events, so these scores measure simulator performance, not real Fiji forecasting accuracy. The input contains present and historical measurements, never future weather or hidden flood labels. This table compares four actual models. Recall and PR-AUC matter more than accuracy when floods are rare. The replay shows a probability, a provisional risk category and a timestamp. Future Fiji MET integration requires licensed readings, compatible units and geography, real event labels, retraining and calibration, and independent geographical and prospective validation before operational use.”

Source trail: `ml/synthetic/{generate,features,train}.py`; `ml/runs/synthetic-v1/metrics.csv`; `ml/artifacts/synthetic/v1/model_metadata.json` (checksum verified); `backend/app/services/{v1_features,v1_model,prediction_generation}.py`. No genuine forecasting or scheduled automatic prediction is verified.

04 · DATABASE / LOCATIONS

Flood Mapping

Status: point map and registry implemented; advanced spatial overlays are absent

Purpose and where to find it

The map shows where monitored places are and what information is stored for each one. Users can select a location and inspect its risk, weather and approved alerts. Open **Weather map** (`/weather-map`); sign-in is not required.

How it works

1. `GET /api/locations` reads active registry rows and their latest operational/sample prediction. Slugs identify map markers; numeric IDs link stored weather and alerts.
2. React-Leaflet places colour-coded markers over OpenStreetMap tiles. Search, division and risk filters narrow the view.
3. Selecting a marker fetches weather separately; approved alerts are matched by location ID. Source, model and stale labels explain what each record means.

Live demonstration

1. Show all **12 locations** and four divisions. Pan and zoom; select the Nausori marker or location row.
2. Search **Nausori**: confirm one visible result and one marker. Clear search, filter **Western**, then try a risk filter.
3. Point out the probability, timestamp, sample/synthetic provenance, stored weather and approved-alert section. Show “No prediction” where applicable.
4. For a signed-in user, open **My overview → My saved locations**. Explain save/remove controls. Location administration uses Swagger `POST /api/admin/locations` and `PATCH /api/admin/locations/{id}`; there is no dedicated admin location form.

Monitored places

Central: Suva, Nausori, Navua. **Western:** Nadi, Lautoka, Ba, Sigatoka, Rakiraki. **Northern:** Labasa, Savusavu. **Eastern:** Levuka, Vunisea (Kadavu).

Success checks and limits

Verified in a browser: 12 markers; Nausori search reduces to one. PostgreSQL confirms the same active IDs and coordinates. Admin API tests cover authorization, valid coordinates, conflicts and deactivation. **Partial SRS scope:** locations use latitude/longitude columns, not PostGIS geometry; only `plpgsql` is installed. There are no inundation polygons, flood-depth surfaces or live push overlays. Tile access needs connectivity. The normal map reads `predictions`; synthetic forecasts remain in their separate replay view.

One-minute lecturer script

“This map uses the existing PostgreSQL location registry rather than a second set of invented monitoring points. We have twelve locations covering four divisions. Selecting Nausori connects its marker to the same location ID used by weather and alerts. Search and filters help users find affected places, while timestamps and source labels prevent old or sample records being mistaken for current forecasts. Administrators can maintain locations through protected APIs, and users can save locations separately. This is a point-based risk map. Although PostGIS appears in the design documents, this installation does not implement PostGIS flood-area or inundation layers.”

Source trail: React-Leaflet/Leaflet, OpenStreetMap, FastAPI and PostgreSQL. `frontend/src/pages/WeatherMapPage.tsx`; `components/map/{WeatherMap,LocationDetailCard}.tsx`; `backend/app/api/routes/locations.py`; `services/location_service.py`. Tables: `locations`, `predictions`, `saved_locations`.

05 · QA / DOCUMENTATION

Alert System & Testing

Status: lifecycle and browser pipeline tested; real desktop delivery is unverified

Purpose and two separate flows

Alerts turn reviewed risk information into clear messages. Administrators retain control over publication; synthetic demonstrations are kept separate from operational records.

Operational: saved prediction/manual draft → PENDING → APPROVED or REJECTED; approved alerts can be SUPPRESSED. Only approved, unexpired alerts at active locations are public.
Synthetic: V1 research output → isolated PENDING review → synthetic approval/feed; never the operational outbox.

Where and demonstration steps

1. Open **Community alerts** (`/alerts`) and show the existing approved messages. MET admins use **Manage alerts** (`/admin/alerts`) to review/edit records. Do not create a public warning merely for this evaluation.
2. Open **Synthetic replay** and show its existing approved alert. Creation/approval is **currently disabled** by `SYNTHETIC_DEMO_ENABLED=false`. On a separately prepared demo instance, enable that gate, load a HIGH/VERY_HIGH result, create pending review, then approve it.
3. For notification behavior, show account opt-in and permission states. In development, after permission is granted, **Test Browser Notification** is available. Count it as verified only if a real desktop popup is observed.
4. Use **My overview → My notification history** for in-app delivery records. Denied/unsupported permission leaves website alerts available.

Rules and success checks

Operational expiry: approval sets a 24-hour lifetime by default. Older NULL-expiry records are exceptions.
Cooldown: automatic same-location recommendations normally reuse a recent alert for six hours; greater severity can escalate. Manual and demo/sample rules differ; these rules do not define the isolated synthetic lifecycle.
Delivery: approved subscribed/opted-in users receive outbox items. The browser polls about every 30 seconds, claims an item, rechecks eligibility and records a receipt. No service worker, Web Push, SMS, email or closed-browser delivery is implemented.

Verification status

Lifecycle, authorization, expiry/cooldown, opt-in, retry and deduplication have automated tests. Two existing outbox rows report DELIVERED, but that is a browser receipt—not independent proof of an OS popup. This audit did not request permission or display notifications. Public and synthetic feeds responded successfully; no new alert was created.

One-minute lecturer script

“There are two alert paths. Operational alerts have administrator review, publication, expiry and suppression. Synthetic alerts are isolated research records and cannot enter the operational notification outbox. The current demonstration gate is off, so I will show existing records and the passing lifecycle tests rather than pretend a new alert was generated. Browser alerts also need account opt-in, saved locations, permission and an open tab. A delivered receipt means the browser reported display; it does not prove the person read it. We can test a desktop popup only on a permitted demo browser and report honestly if it does not appear.”

Source trail: `backend/app/services/{alert_service,synthetic_alert_service,notification_outbox}.py`; `frontend/src/alerts/{useBrowserAlerts,outboxDelivery}.ts`. APIs: `/api/admin/alerts`, `/api/alerts/active`, `/api/synthetic/*`, `/api/me/subscriptions`, `/notification-preferences`, `/notifications` under `/api/me`.

LIVE WALKTHROUGH

End-to-End Demo Checklist

A practical sequence using the actual current dataset and controls

Start the existing prepared installation

Requires existing PostgreSQL schema/data, local model artifacts, approved accounts and configured environment files. This audit did not initialize or migrate them. Use two terminals; avoid duplicate servers on ports 8000 and 5173.

```
# Terminal 1 — from the project root
cd backend
.venv/bin/python -m uvicorn app.main:app --reload \
  --host 127.0.0.1 --port 8000

# Terminal 2 — from the project root
cd frontend
npm run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

Open `http://127.0.0.1:5173`; API docs: `http://127.0.0.1:8000/docs`; health: `/api/health`. Observed environment: Python 3.14.7, Node 26.5.0. Dependencies are already installed; use each project's lockfile/requirements if setup is needed. Keep credentials outside the PDF and source control.

End-to-end lecturer checklist

Step	Action and honest success criterion
1 · Account	Sign in with an approved account. A MET admin is needed for analytics/review. No credentials are supplied here.
2 · Dashboard	Show 12 monitored locations and current alert counts. Identify samples and timestamps; counts can change with expiry.
3 · Weather	Choose Nausori on the map. Show synthetic/stale readings; genuine API result is no_data.
4 · AI	Model performance → V1 test. Then /simulation → 2024-07-30T01:00 UTC → Load synthetic replay; Nausori ≈72.8%.
5 · Geography	Search/filter map; explain four divisions and matching location IDs. Normal map samples and replay scores are separate.
6 · Alert review	Show existing approved operational/synthetic alerts. New synthetic approval is blocked by the current gate; demonstrate only on an isolated, pre-enabled copy.
7 · Notifications/tests	Show history and permission status. Desktop popup remains conditional. Present the fresh test evidence on page 8.

Avoid a common demo mismatch

The full replay archive includes **30 July 2024**, while the canonical stored-weather snapshot ends on **10 July 2024**. A July 30 call to the stored-weather generation API can legitimately return `DATA_UNAVAILABLE`. Read-only inference from the existing July 10 snapshot produced **0.2507% LOW** for Nausori. It was not persisted and did not create an alert.

If a step is unavailable

Show the actual empty/error/disabled state and the verified tests or API schema. Do not change data or settings to conceal a blocker; separate preparation needs authorization.

EVIDENCE & QUESTIONS

Verification and Limitations

A passing test is evidence of its tested scope—not proof of real-world forecasting

Fresh verification · 21 October 2026

Check	Observed result / scope
Backend API suite	**271 passed, 9 skipped** · 43.78 s; SQLite-isolated API/service tests.
ML suite	**46 passed, 1 skipped** · 20.94 s; causal features, splits, labels and research workflow.
Frontend suite	**155 passed** in 22 files · 7.06 s; jsdom component/integration tests.
Frontend quality	**oxlint passed; TypeScript + Vite production build passed.**
Live read checks	Health, 12 locations, four divisions, weather, V1 replay and alert feeds responded. Protected analytics returned 401 without auth.
Public browser checks	12 map markers; Nausori search → one; replay timestamp/source/72.8% visible; admin page redirects guests.

Skipped tests: six PostgreSQL location-migration tests lack an explicit test database; three Training-02 compatibility tests lack historical artifacts. The ML skip requires an absent historical audit artifact. PostgreSQL migration/concurrency suites were **not run** because they create disposable schemas. The deprecated India-data suite was not used. One non-failing Starlette/httpx deprecation warning occurred.

Important limitations to tell the lecturer

1. The SRS/SAD propose external weather, PostGIS and timely push-style updates. Actual delivery is a partial academic implementation: no real provider, no PostGIS geometry, no automatic genuine Fiji forecasting and no closed-browser push.
2. The normal map currently has only sample prediction records; 2,100 canonical weather rows are historical synthetic data. Newly generated V1 predictions use separate synthetic tables.
3. Synthetic alert creation is off. Existing approved alerts can be viewed; live signed-in admin writes and OS notification delivery were not exercised.
4. Synthetic accuracy is not real-world forecast skill. V1 event-level recall differs from positive-hour recall; V2/V3 results cannot be presented as deployed V1 performance.

Likely lecturer questions

Why not claim 98% flood-detection success? That is accuracy under class imbalance; V1 positive-hour recall is 29.59%.
Is the newest research winner deployed? No. V1 CatBoost remains served; V2/V3 are read-only research.
Can the model use Fiji MET readings now? Genuine inference is explicitly blocked until compatible geography, units, event labels and independent validation exist.
Does “DELIVERED” prove a user saw a warning? No. It records a browser receipt, not human acknowledgement.
Are the pages fully live? They use APIs, but data can be synthetic, sampled, stale or empty. Updates use requests/polling, not a WebSocket stream.

Source trail: Requirements reviewed: `docs/SRS_Group13.pdf`, `docs/SAD_Group13-1.pdf`. Supporting source/evidence index: `docs/IS314_Five_Functionalities_Audit_Evidence.md`. Test commands are recorded there. No screenshots or externally assumed performance claims are used in this guide.